



YOUR **AI** IS TOO MUCH **EXPENSIVE**

Top 10 Techniques to Optimize AI Models
along with **Code Samples**



Chandra Sekhar
AI Engineer & Educator

Why Optimize AI Models?

AI optimization is essential for building production-ready systems that are cost-effective, performant, and scalable.

Without optimization, AI models can become **prohibitively expensive** and **unable to scale** to meet real-world demands.

💰 REDUCE COSTS

- Lower compute and infrastructure expenses
- Reduce API and cloud service costs
- Minimize memory and storage requirements

⚡ IMPROVE PERFORMANCE

- Faster inference and response times
- Higher throughput and efficiency
- Better resource utilization
- Enhanced user experience



10 Top Techniques to Optimize AI Models

1

Model Quantization

2

Low-Rank Adaptation
(LoRA)

3

Fine-Tuning

4

Pruning

5

Batching

6

Gradient Checkpointing

7

Model Compression

8

Optimized Hardware

9

Caching

10

Prompt Engineering

Master these techniques to reduce costs, improve performance, and scale efficiently



Model Quantization

What it is: Reduce model weight precision from 32-bit floating point to 8-bit integers.

Benefits: Lower memory usage, faster computations, and reduced costs while maintaining performance.

CODE SNIPPET

```
# Option 1: 8-bit quantization with bitsandbytes
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    "model_name",
    load_in_8bit=True,
    device_map="auto"
)

# Option 2: Post-training dynamic quantization
import torch

quantized_model = torch.quantization.quantize_dynamic(
    model,
    {torch.nn.Linear},
    dtype=torch.qint8
)
```



Low-Rank Adaptation (LoRA)

What it is: Adapt small, low-rank layers in neural networks for task-specific fine-tuning without modifying the entire model.

Benefits: Minimizes memory and computational demands while customizing models effectively.

CODE SNIPPET

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM

# Load base model
model = AutoModelForCausalLM.from_pretrained("model_name")

# Configure LoRA
lora_config = LoraConfig(
    r=8, # rank
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.1
)

# Apply LoRA
model = get_peft_model(model, lora_config)
```



Fine-Tuning

What it is: Adapt pretrained models to specific tasks using domain-specific data.

Benefits: Avoids training from scratch, saves time and cost while achieving specialized performance.

CODE SNIPPET

```
from transformers import
AutoModelForSequenceClassification, Trainer

# Load pretrained model
model =
AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2
)

# Fine-tune on custom data
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset
)
trainer.train()
```



Pruning

What it is: Remove unnecessary neurons or weights from trained models while maintaining accuracy.

Benefits: Significantly reduces model size and inference time, lowering operational costs.

CODE SNIPPET

```
import torch
import torch.nn.utils.prune as prune

# Magnitude pruning
prune.l1_unstructured(
    module=model.layer,
    name='weight',
    amount=0.3 # Prune 30% of weights
)

# Structured pruning
prune.ln_structured(
    module=model.conv1,
    name='weight',
    amount=0.5,
    n=2,
    dim=0
)
```



Batching

What it is: Combine multiple requests into single operations to maximize hardware utilization.

Benefits: Increases computational efficiency, reduces GPU/CPU idle time, and significantly lowers costs.

CODE SNIPPET

```
import torch

# Dynamic batching for inference
def batch_inference(requests, model, batch_size=32):
    results = []
    for i in range(0, len(requests), batch_size):
        batch = requests[i:i + batch_size]
        inputs = tokenizer(batch, padding=True,
return_tensors="pt")

        with torch.no_grad():
            outputs = model(**inputs)
            results.extend(outputs)

    return results
```



Gradient Checkpointing

What it is: Trade computation for memory by recomputing activations during backpropagation instead of storing them all.

Benefits: Significantly reduces memory usage during training, enabling larger models or batch sizes on limited hardware.

CODE SNIPPET

```
import torch
from torch.utils.checkpoint import checkpoint

# Enable gradient checkpointing for a model
model = MyModel()
model.gradient_checkpointing_enable()

# Or manually checkpoint specific layers
class CheckpointedModel(torch.nn.Module):
    def forward(self, x):
        # Checkpoint intermediate layers
        x = checkpoint(self.layer1, x)
        x = checkpoint(self.layer2, x)
        return self.layer3(x)

# Memory saved: ~50-80% at cost of ~20% slower training
```



Model Compression

What it is: Reduce model size through pruning, quantization, or distillation techniques.

Benefits: Lowers memory requirements, accelerates inference, and reduces storage costs.

CODE SNIPPET

```
# Combined compression approach
from torch.quantization import quantize_dynamic
import torch.nn.utils.prune as prune

# 1. Prune the model
prune.global_unstructured(
    parameters=[(model.layer, "weight")],
    pruning_method=prune.L1Unstructured,
    amount=0.2
)

# 2. Quantize
compressed_model = quantize_dynamic(
    model,
    {torch.nn.Linear},
    dtype=torch.qint8
)
```



Optimized Hardware

What it is: Use specialized devices (TPUs, GPUs, ASICs) designed for AI workloads.

Benefits: Faster computation, reduced energy consumption, and lower long-term costs.

CODE SNIPPET

```
import torch

# GPU optimization
device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
model = model.to(device)

# Mixed precision training (GPU)
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()
for data, target in dataloader:
    with autocast():
        output = model(data)
        loss = criterion(output, target)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```



Caching

What it is: Store frequently used computation results to avoid recalculating them.

Benefits: Saves computational resources, reduces latency, and cuts costs for repetitive queries.

CODE SNIPPET

```
import redis
import json

# Initialize Redis cache
cache = redis.Redis(host='localhost', port=6379, db=0)

def cached_inference(prompt, model):
    # Check cache first
    cached_result = cache.get(prompt)
    if cached_result:
        return json.loads(cached_result)

    # Generate if not cached
    result = model.generate(prompt)

    # Store in cache (24 hour expiry)
    cache.setex(prompt, 86400, json.dumps(result))
    return result
```



Prompt Engineering

What it is: Craft input prompts to guide models toward accurate and efficient outputs.

Benefits: Reduces token usage, optimizes API costs, and ensures high-quality results.

CODE SNIPPET

```
# Effective prompt template
def create_optimized_prompt(task, context, examples=None):
    prompt = f"""Task: {task}

Context: {context}

Instructions:
- Be concise and specific
- Use structured format
- Focus on key information

"""
    if examples:
        prompt += "Examples:\n"
        for ex in examples:
            prompt += f"Input: {ex['input']}\nOutput: {ex['output']}\n\n"

    return prompt
```



Want more content like this?



Tap that follow button and
stay in the loop!



Like



Comment



Share



Save